



คู่มือบทวนโครงงาน Portfolio

Microservices — NestJS + React + MongoDB + Omise

Panapol Sukcharoen

พิมพ์ออกมาอ่านก่อนสัมภาษณ์ · commerce-api / portfolio-readmes

Fruit Shop — ภาพรวมทั้งโปรเจกต์

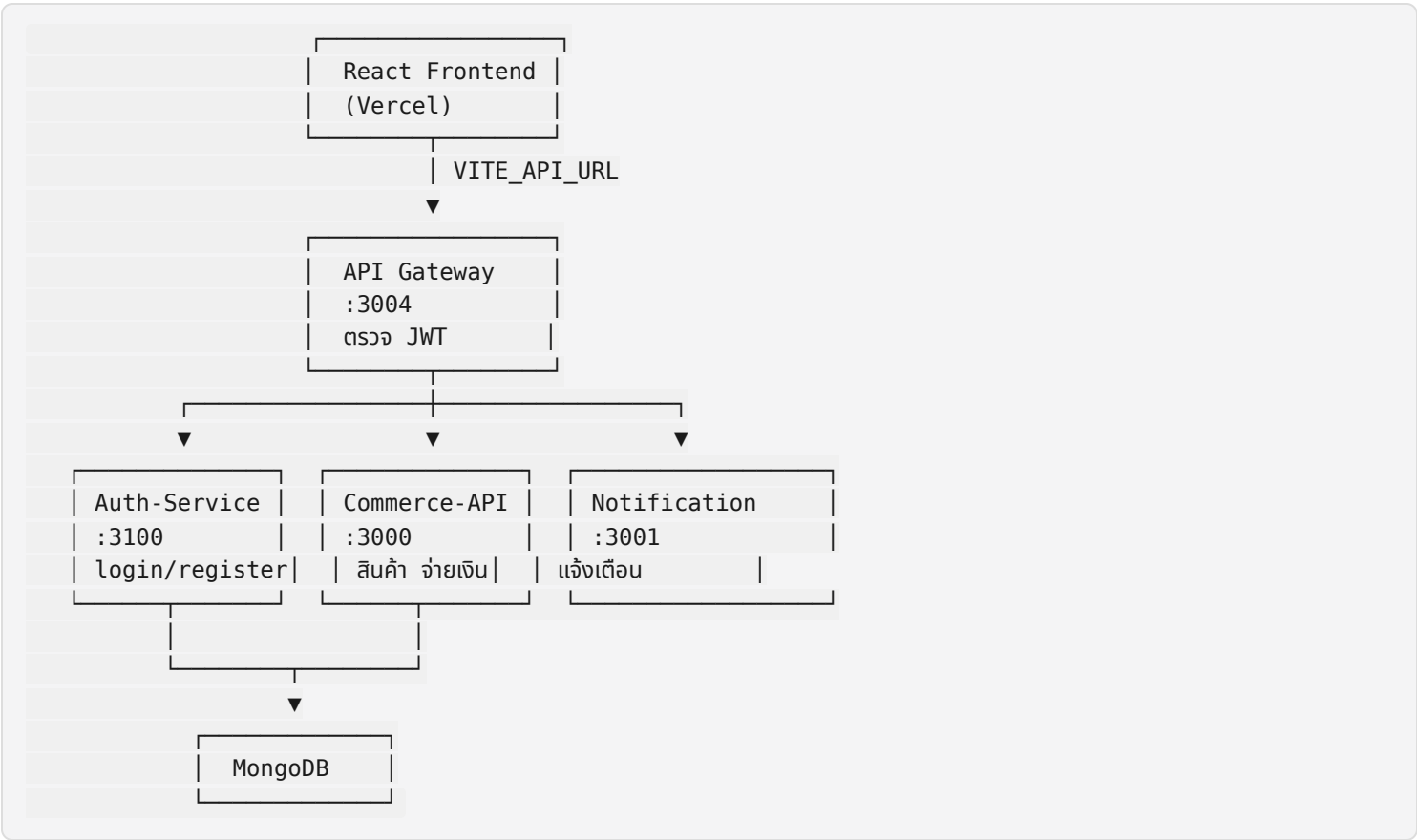
เอกสารนี้เขียนให้ **ตัวเราเองอ่านทบทวน** ก่อนสัมภาษณ์

ไม่ต้องจำทุกบรรทัด — จำ **flow** กับ **ทำไมแยก service** พอ

ระบบทำอะไร?

เว็บขายผลไม้: สมัครสมาชิก → ดูสินค้า → ใส่ตะกร้า → checkout → จ่ายบัตร → ได้แจ้งเตือน

แผนภาพ (วาดในใจแบบนี้)



แต่ละ repo หน้าที่จะไร (หนึ่งประโยค)

Repo	หน้าที่สั้นๆ
frontend	หน้าจอให้ user กด — เรียก API แแค่ Gateway
Api-Gateway	ประตูเดียว + แปลง JWT เป็น headers ส่งต่อ
Auth-Service	สมัคร / login / refresh — ออก JWT
commerce-api	สินค้า ตะกร้า ออเดอร์ Omise
notification-service	เก็บและแสดงแจ้งเตือน

ลำดับรันบนเครื่อง (local)

- MongoDB (Atlas หรือ local)
- Auth-Service :3100
- Commerce-API :3000 (+ yarn seed ครั้งแรก)
- Notification :3001
- Api-Gateway :3004
- Frontend yarn dev → localhost:5173

Secret ที่ต้อง ตรงกัน ข้าม repo

ตัวแปร	ใช้ที่ไหนบ้าง
JWT_SECRET	Auth, Gateway (verify), Commerce (เรียก notification)
GATEWAY_SECRET	Gateway (ส่ง) + Commerce (รับ)
VITE_OMISE_PUBLIC_KEY	Frontend
OMISE_SECRET_KEY	Commerce (charge) — คู่กับ public key

คำถามสัมภาษณ์ที่เตรียมไว้

Q: ทำไมใช้ microservices?

A: แยกหน้าที่ชัด — auth เปลี่ยนบ่อยไม่กระทบ commerce, deploy แยกได้, ฝึก pattern จริงใน portfolio

Q: Gateway ทำอะไร?

A: Frontend เรียกทีเดียว, verify JWT, ส่ง x-user-id/email/role + x-gateway-secret ไป commerce

Q: กันคนอื่นจ่ายออเดอร์เราได้ไหม?

A: `charge()` เช็ค `order.userId === req.user.id` → 403 ถ้าไม่ใช่เจ้าของ

Q: 401 vs 403?

A: 401 ยังไม่ auth / token พัง — 403 auth แล้วแต่สิทธิ์ไม่พอ

ลิงก์ README แต่ละตัว

- Commerce API
- API Gateway
- Auth Service
- Frontend
- Notification Service

Commerce API — ร้านผลไม้ (Fruit Shop)

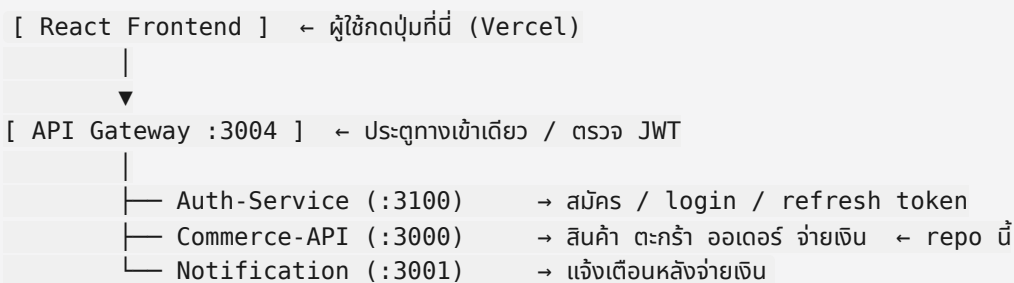
Repo นี้ทำอะไร?

เป็นส่วน หลังบ้านร้านค้า — จัดการสินค้า ตะกร้า ออเดอร์ และรับเงินผ่าน Omise

ไม่ใช่ ตัว login หลัก (login อยู่ที่ Auth-Service ผ่าน Gateway)

Demo Frontend: <https://fruit-shop-frontend-six.vercel.app>

ภาพรวมระบบทั้งหมด (จำแบบนี้)



ทำไมไม่ให้ Frontend ยิงตรง Commerce?

- ซ่อน URL จริงของแต่ละ service
- ตรวจ JWT ที่ Gateway จุดเดียว
- Commerce รู้ว่า request มาจาก Gateway จริง (ผ่าน `GATEWAY_SECRET`)

ใน repo นี้มีอะไรบ้าง

โฟลเดอร์	ทำอะไร
<code>src/products</code>	CRUD สินค้า (admin เท่านั้นที่สร้าง/แก้/ลบ)
<code>src/cart</code>	ตะกร้า — เพิ่ม/ลบสินค้า, stock ลดตอนใส่ตะกร้า
<code>src/orders</code>	checkout สร้างออเดอร์ <code>pending</code>
<code>src/payments</code>	จ่ายผ่าน Omise + webhook สำรอง
<code>src/notifications</code>	เรียก Notification Service หลังจ่ายสำเร็จ
<code>src/auth</code> + <code>src/users</code>	guard รับ user จาก Gateway headers (ไม่ใช่ login หลัก)
<code>src/seed.ts</code>	ใส่ admin + สินค้าตัวอย่างใน DB

Flow ที่ควรเข้าใจ (สำหรับสัมภาษณ์)

1) Login (ไม่ได้เกิดใน repo นี้โดยตรง)

```
Frontend → Gateway → Auth-Service
→ ได้ access_token + refresh_token
→ Frontend เก็บใน localStorage
```

2) ชื่อของ

```
GET /products      → ดูลิสต์สินค้า (ไม่ต้อง login)
POST /cart/add      → ใส่ตะกร้า (ต้อง login) + stock an
POST /orders/checkout → สร้าง order status: pending
```

ถ้ามี pending order อยู่แล้ว → **ไม่สร้างซ้ำ** (คืนอันเดิม)

3) จ่ายเงิน

```
Frontend สร้าง Omise token จากบัตร (vault.omise.co)
→ POST /payments/charge { orderId, token }
→ Commerce เช็คว่า order เป็นของ user นี้จริง (กัน IDOR)
→ Omise ตัดเงิน → order = paid → ล้างตะกร้า
→ ส่ง notification ไป Notification Service
```

4) รู้ว่าใครเรียก API (Guards)

```
JwtAuthGuard
1. เช็ค x-gateway-secret (ถ้ามี GATEWAY_SECRET ใน .env)
2. อ่าน x-user-id, x-user-email, x-user-role จาก Gateway
3. ใส่ req.user ให้ controller ใช้
```

```
RolesGuard + @Roles('admin')
→ เช็ค role จาก req.user
```

- **401** = ยังไม่ login / token ผิด / secret ผิด
- **403** = login แล้วแต่ role ไม่พอ (เช่น user อยากลบสินค้า)

วิธีรันบนเครื่อง

ต้องมีก่อน

- Node.js 20+
- MongoDB (Atlas หรือ local)
- Omise test keys
- **Gateway + Auth รันด้วย** ถ้าจะทดสอบผ่าน Frontend จริงๆ

ขั้นตอน

```
git clone https://github.com/panapolll/commerce-api.git
cd commerce-api
yarn install
cp .env.example .env # แก้ค่าให้ครู
yarn seed # สร้าง admin + สินค้า (ต้องมี SEED_ADMIN_* ใน .env)
yarn start:dev # รับที่พอร์ต 3000
```

ตัวแปรใน .env (อธิบายทีละตัว)

ตัวแปร	ทำไมต้องมี
MONGO_URI	เชื่อม MongoDB เก็บสินค้า ตะกร้า ออเดอร์
JWT_SECRET	ใช้ sign token ตอน commerce เรียก notification (ต้องตรงกับ Auth)
GATEWAY_SECRET	คู่กับ Gateway — กันยิง commerce ตรงโดยไม่ผ่าน Gateway
OMISE_PUBLIC_KEY / OMISE_SECRET_KEY	รับเงิน test mode
NOTIFICATION_SERVICE_URL	URL ของ notification service (เช่น http://localhost:3001)
SEED_ADMIN_EMAIL / SEED_ADMIN_PASSWORD	ใช้ตอน yarn seed เท่านั้น

API สำคัญ (ผ่าน Gateway ไม่มี prefix /api)

Method	Path	ใครใช้ได้
GET	/products	ทุกคน
POST	/products	admin
GET	/cart	login
POST	/cart/add	login
POST	/orders/checkout	login
GET	/orders/me	login
POST	/payments/charge	login (เจ้าของ order)

Repo ที่เกี่ยวข้อง

Service	GitHub
Frontend	fruit-shop-frontend
API Gateway	Api-Gateway
Auth Service	Auth-Service
Notification	notification-service

README ภาษาไทยทุก service อยู่ในโฟลเดอร์ `portfolio-readmes/`

สิ่งที่แก้/เรียนรู้จากโปรเจกต์นี้

- แยก microservice ตามหน้าที่ (auth ≠ commerce)
- ปิด endpoint เปิดโล่ง (`POST /users/:createUsers`)
- กัน IDOR ตอนจ่ายเงิน (เช็คเจ้าของ order)
- `GATEWAY_SECRET` คู่กัน Gateway ↔ Commerce
- Omise: token ที่ frontend, charge ที่ backend
- Notification ไม่ให้พัง flow จ่ายเงิน (try/catch + log)

ผู้พัฒนา

Portfolio project — Panapol Sukcharoen (career changer → full-stack / backend)

Commerce API — คู่มือสั้น (อ่านเอง)

ดู README หลักใน root repo ด้วย — ไฟล์นี้สรุปซ้ำแบบง่าย

จำสามคำ

สินค้า → ตะกร้า → ออเดอร์ → จ่ายเงิน → แจ้งเตือน

Port

3000

คำสั่ง

```
yarn install
cp .env.example .env
yarn seed
yarn start:dev
```

Guard สำคัญ

- `JwtAuthGuard` — รับ user จาก Gateway headers + เช็ค `GATEWAY_SECRET`
- `RolesGuard` — admin เท่านั้นสำหรับ CRUD สินค้า

อย่าเปิด endpoint แบบนี้อีก

- `POST /users/:createUsers` ไม่มี guard → ลบแล้ว
- สมัคร user ผ่าน Auth-Service เท่านั้น

API Gateway — คู่มืออ่านเอง

Gateway คืออะไร?

ประตูทางเข้าเดียว ระหว่าง Frontend กับ backend หลายตัว

Frontend ไม่ต้องรู้ว่า Auth อยู่พอร์ตไหน Commerce อยู่พอร์ตไหน — รู้แค่ URL ของ Gateway

ทำอะไรบ้าง?

- รับ request จาก Frontend (Vercel / localhost)
- route ไป service ที่ถูก (/auth/* → Auth, /products → Commerce)
- ถ้า route ต้อง login → GatewayAuthGuard verify JWT
- ส่งต่อไป Commerce พร้อม headers:
 - x-user-id , x-user-email , x-user-role
 - x-gateway-secret (คู่กับ Commerce .env)

แผนภาพ



ไม่มี /api นำหน้า — `setGlobalPrefix('api')` ถูก comment ไว้

วิธีรับ

```
git clone https://github.com/panapollll/Api-Gateway.git
cd Api-Gateway
yarn install
cp .env.example .env
yarn start:dev
```

Branch หลัก: **master** (ไม่ใช่ **main**)

Environment

ตัวแปร	ตัวอย่าง	หมายเหตุ
PORT	3004	
AUTH_SERVICE_URL	http://localhost:3100	
COMMERCE_API_URL	http://localhost:3000	
NOTIFICATION_SERVICE_URL	http://localhost:3001	
JWT_SECRET	ตรงกับ Auth	ใช้ verify token
GATEWAY_SECRET	สตริ่งลับเดียวกับ Commerce	ส่งใน x-gateway-secret

CORS

อนุญาต:

- http://localhost:5173
- https://fruit-shop-frontend-six.vercel.app

credentials: true สำหรับ cookie/credentials ถ้าใช้ในอนาคต

ไฟล์สำคัญ

ไฟล์	ทำอะไร
commerce-proxy.service.ts	ส่ง request ไป Commerce + buildHeaders()
auth-proxy/*	login register refresh
gateway-auth.guard.ts	ดึง JWT จาก Authorization header

Push จำไว้

```
git push origin master
```

หน้าที่

จัดการ **ตัวตนผู้ใช้** — สมัคร, login, refresh token, logout

ไม่เก็บสินค้า ไม่เก็บตะกร้า — แค่ user + password (hash) + refresh token ใน DB

Flow login (จําแบบนี้)

1. User ส่ง email + password
 2. bcrypt เทียบ password
 3. ออก access_token (15 นาที) + refresh_token (1 ชม.)
 4. เก็บ refresh_token (hash) ใน MongoDB
 5. ส่ง tokens กลับ Frontend

Token หมดอายุ → เรียก POST /auth/refresh ด้วย `userId` + `refreshToken`

ValidationPipe (ข้อ 10 ที่แก้)

`main.ts` เปิด `ValidationPipe` global แล้ว

`RefreshTokenDto` แก่จาก `@IsEmail()` ผิด field เป็น:

- `refreshToken` → `@IsString()` + `@IsNotEmpty()`
- `userId` → `@IsMongoId()`

API

Method	Path	หมายเหตุ
POST	/auth/register	สมัคร user
POST	/auth/login	ได้ tokens
POST	/auth/refresh	ต่ออายุ access
POST	/auth/logout	ต้อง Bearer token
POST	/auth/verify	Gateway ใช้ตรวจ token

วิธีรับ

```
git clone https://github.com/panapolll/Auth-Service.git
cd Auth-Service
yarn install
cp .env.example .env
yarn start:dev
```

Port: **3100**

Environment

ตัวแปร	หมายเหตุ
MONGODB_URI	MongoDB ของ Auth (แยกจาก Commerce ใต้)
JWT_SECRET	ต้องตรง กับ Gateway
REFRESH_TOKEN_SECRET	sign refresh token
PORT	3100

Live

<https://auth-service-7xty.onrender.com>

สัมภาษณ์

Q: ทำไมไม่ให้ Commerce login เอง?

A: Single responsibility — Auth ดูแล identity อย่างเดียว เปลี่ยน policy login ไม่กระทบร้านค้า



Frontend — คู่มืออ่านเอง

React + Vite — หน้าที่ทำให้ user ใช้งานจริง

Repo จริง: `fruit-shop-frontend` หรือ `ReactSeries/frontend` (แล้วแต่เครื่อง)

Deploy: Vercel → <https://fruit-shop-frontend-six.vercel.app>

หลักการสำคัญ

Frontend เรียก API แยก **Gateway** — ไม่ยิงตรง Commerce หรือ Auth

```
// config.ts
export const GATEWAY_URL = import.meta.env.VITE_API_URL ?? "http://localhost:3004";
```

Production บน Vercel ต้องตั้ง:

```
VITE_API_URL=https://your-gateway.onrender.com
VITE_OMISE_PUBLIC_KEY=pkey_test_xxxxx
```

หลังเพิ่ม env ต้อง **Redeploy** — แก้ในเครื่องอย่างเดียว Vercel ไม่เปลี่ยน

หน้าหลัก

Path	หน้า
<code>/login</code>	เข้าสู่ระบบ
<code>/register</code>	สมัคร (validate ภาษาไทย)
<code>/products</code>	ดูสินค้า
<code>/cart</code>	ตะกร้า
<code>/payment</code>	จ่าย Omise
<code>/orders</code>	ออเดอร์ของฉัน
<code>/notifications</code>	แจ้งเตือน

Omise (จ่ายเงิน)

- `index.html` โหลด `https://cdn.omise.co/omise.js`
- `PaymentPage` เรียก `Omise.createToken('card', {...})`

- ได้ token ส่งไป POST /payments/charge ผ่าน Gateway

บัตรทดสอบ: 4242 4242 4242 4242

ปี: ต้อง 4 หลัก เช่น 2028 (ไม่ใช่ 28)

Error vault.omise.co/tokens 400 → มักเป็น public key ผิด หรือปีบัตรผิด

Token เก็บที่ไหน

localStorage — access_token, refresh_token, userId

Axios interceptor ใน client.ts แบบ Authorization: Bearer ... และ refresh อัตโนมัติเมื่อ 401

วิธีรัน

```
yarn install
yarn dev
```

เปิด http://localhost:5173

Push ขึ้น Vercel

```
git add .
git commit -m "... "
git push origin main
```

รอ Vercel deploy → hard refresh (Ctrl+Shift+R)



Notification Service — คู่มืออ่านเอง

หน้าที่

เก็บ แจ้งเตือนในแอป — เช่น "ชำระเงินสำเร็จ"

ไม่ส่ง email / SMS — แค่ in-app notification ที่ user เปิดดูใน Frontend

ใครเรียกใช้?

ใคร	เมื่อไหร่
Commerce-API	หลัง Omise charge สำเร็จ → <code>POST /notifications</code> (service-to-service)
Frontend	ผ่าน Gateway → <code>GET /notifications/me</code> , mark read, au

Flow หลังจ่ายเงิน

```
Commerce charge() สำเร็จ
→ notificationsService.sendPaymentSuccess()
→ sign JWT ชื่อ role admin (service account)
→ POST notification-service/notifications
→ บันทึก MongoDB
```

```
User เปิดหน้า Notifications
→ Gateway → GET /notifications/me
→ เห็น "ชำระเงินสำเร็จ 🇹🇭"
```

ถ้า notification ล้ม → **จ่ายเงินยังสำเร็จ** (commerce ใช้ try/catch ไม่ให้พัง flow)

วิธีรับ

```
git clone https://github.com/panapolll/notification-service.git
cd notification-service
yarn install
cp .env.example .env
yarn start:dev
```

Port: **3001**

Environment

ตัวแปร	หมายเหตุ
MONGODB_URI	เก็บ notifications
JWT_SECRET	ตรงกับ Auth — verify Bearer จาก user
PORT	3001

Commerce ต้องมี NOTIFICATION_SERVICE_URL ขึ้นมาที่นี้

API (ผ่าน Gateway)

Method	Path	ใคร
GET	/notifications/me	user login
GET	/notifications/unread-count	user login
PATCH	/notifications/mark-all-read	user login
PATCH	/notifications/:id/read	user login
DELETE	/notifications/:id	user login
POST	/notifications	admin / service (commerce เรียก)

สัมภาษณ์

Q: ทำไมแยก service?

A: แจ้งเตือนอาจขยายเป็น push/email ที่หลัง ไม่ต้องแก้ commerce

Q: ทำไม commerce sign JWT เอง?

A: POST สร้าง notification ต้อง admin — commerce เป็น trusted internal caller